Oppo flagship returns
The Find series is coming back, for the first time since 2014, with the Find X which was unveiled at the Louvre. **https://dgit.in/Jul18-42-1**

A drone drops crime rate
A DJI drone with the Mexican Police reduced crime rates by 10% over 4 months, assisting 50 arrests and more. **https://dgit.in/Jul18-42-2**

slashy strings that is started with `$/` and ended with `/$` where the escape character is $ instead of backslash.

In Groovy a string literal can be treated as an array of characters and you can access characters on the basis of the index that is available. Also you can use other methods that come with the normal Java String class. You can provide a range to fetch a range of values from a String. You can get specific characters if you pass them to the implementation. To concatenate Strings you can use either + operator or `.concat` method that is available, as shown in at https://dgit.in/Groovy4, with the output here: https://dgit.in/GrvyOP1.

Now coming to some funny operations that you can do with Groovy. To repeat a String you can use `*` operator. To remove the first occurrence from a String you can use the `-` operator. You can compare two strings with `<=>` operator. This will return -1 if the first string is less in alphabetic order than the second string, 0 if equal and 1 if greater than the second string. Check out an example code snippet here: https://dgit.in/Groovy5 and its output here: https://dgit.in/GrvyOP2.

LISTS, MAPS AND RANGES

First up it is Lists. Groovy uses a comma-separated list of values, surrounded by square brackets, to denote lists. Groovy lists are plain JDK `java.util.List`, as Groovy doesn't define its own collection classes. The concrete list implementation used when defining list literals are `java.util.ArrayList` by default, unless you decide to specify otherwise. To define a list in Groovy just add the objects within a square bracket. Like the following,

```
def listOfPrimes =
[2, 3, 5, 7, 11, 13]
```

To get the value from any index of a list you can use square brackets or `get` method for any index, as shown here: https://dgit.in/Groovy6. To get the size of the list you can call the `size` method that is available within the list interface of Java, as shown here: https://dgit.in/Groovy7. Lists in Groovy can hold a mixed type of values. They don't need to be of same type, as shown here: https://dgit.in/Groovy8. You can add some value at the end of the list by the `add` method or by appending using the `<<` operator, as shown here: https://dgit.in/Groovy9. Concatenation of two lists is possible in Groovy. Just use the `+` operator for concatenation, as shown here: https://dgit.in/Groovy10. To remove an element from a list you can use the `-` operator. But beware that you need to pass another list to the operation to delete one element, as shown here: https://dgit.in/Groovy11. You can also pass an set of ranges to the list argument to get the sublist, as shown here: https://dgit.in/Groovy12. To check if the list is empty or not you can use the method `isEmpty`, as shown here: https://dgit.in/Groovy13. To get the matches from an list you can use the method `intersect`, as shown here: https://dgit.in/Groovy14,

To reverse a list you can use the method `reverse`. To sort a list use sort and to get the last element you can use `pop`, as shown here: https://dgit.in/Groovy15. The output of the above is shown here: https://dgit.in/GrvyOP3.

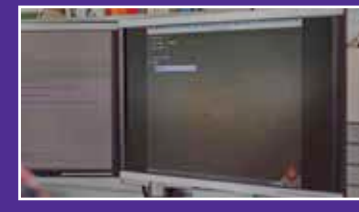
Now Groovy supports the concept of ranges and provides a notation (..) to create ranges of objects, as shown here: https://dgit.in/Groovy16. To get the size you can use `.size` method. To get a value from an index use `.get` method. Check if any range contains any value you can use the `.contain` method. To get the first of the range you can use the `.getFrom` method and lastly to get the last element of any range you can use `.getTo` method. All of the above methods are shown in the code snippet at https://dgit.in/Groovy17 and the outputs that they will generate are shown at: https://dgit.in/GrvyOP4.

Maps are a set of key value pairs. To define maps you can use square brackets then followed by key value pairs. A sample code to explain this is available here: https://dgit.in/Groovy18.

*POINTERS

***The Complete Apache Groovy Developer Course**
Create Groovy applications from scratch, Use the Groovy console, write Groovy applications in IntelliJ and more.
https://dgit.in/GroovyUdemy

***Groovy and Grails Certification Training**
This course teaches Java developers the basics of Groovy and how to use the Grails framework to rapidly create sophisticated web applications..
https://dgit.in/GroovyGrailsCert



***Groovy Fundamentals**
Throughout this course you'll develop a Groovy application that can parse GPS data from an XML file, insert it into a database, and even correlate this data to forecast data retrieved from a REST API.
https://dgit.in/GroovyFundamentals